

Monopoly Deal Card Game Group16 Document

Group Members:

[Yang Yanzhen] [Gong Chen] [Guan Jian] [Zheng Bowei] [Chu Tianze]

1. Introduction

1.1 Game Introduction

Monopoly Deal is a streamlined, strategy-focused card game adapted from the classic Monopoly tabletop experience. Designed for 2–5 players, this version abandons dice rolls and physical boards in favor of card-driven decision-making, emphasizing resource control, tactical card usage, and adaptive responses to opponents' actions. As one of the most popular casual card games globally, it provides an ideal model for practicing object-oriented software development.

This project involves developing a complete, rule-compliant digital version of Monopoly Deal using Java. Through the design and implementation process, the project will apply core software engineering concepts, including modular design, encapsulation, inheritance, and interface-based programming, while simulating real-world game logic in a maintainable and extensible application structure.

1.2 Overview of The Project

This project delivers a standalone Java-based digital Monopoly Deal system that replicates the official 110-card deck, including property cards, money cards, action cards, rent cards, and building upgrade cards. The core victory condition requires players to become the first to complete three full property sets of different colors.

The system strictly implements official game mechanics: each turn follows a fixed “Draw 2 cards, play up to 3 cards” structure, supports property-based payment, asynchronous defense, and hand-size limits. The project also includes a clear graphical user interface (GUI) that visually separates the player's hand, bank area, and property zone to improve usability and interaction clarity.

1.3 Glossary of Key Terms

Bank Area: A dedicated zone where players store money cards and action cards used as currency.

Property Area: A public zone where players place property cards to form complete color sets.

Hand: Private cards held by a player, hidden from other participants.

Draw Pile: The main shuffled deck from which players draw cards at the start of their turn.

Discard Pile: A pile where used action cards, excess hand cards, and expired effects are placed.

Property Set: A full collection of property cards belonging to the same color group, required to win.

Money Card: A card with a fixed monetary value used exclusively for paying fees and rent.

Action Card: A card that triggers special effects such as rent collection, theft, forced trade, or defense.

Building Card: Includes House and Hotel cards, which increase rent values when attached to completed property sets.

Rent Payment: The core economic interaction where a player charges others based on completed property sets.

2. Game Rules Analysis and the Core Logic

2.1 Game Objective & End State

The goal of the game is to be the first player to complete and display **three full property sets of different colors** in their Property Area. Once a player meets this condition, the game immediately ends, and that player is declared the winner. No further actions can be performed once the victory condition is triggered.

2.2 Card Classification and Functional Logic

The game uses a fixed deck of 110 cards divided into four functional categories:

1. **Property Cards:** Placed in the Property Area to form color sets. Wildcards can substitute for any color.
2. **Money Cards:** Used as currency for payments. Denominations include 1M, 2M, 3M, 4M, 5M, and 10M.
3. **Action Cards:** Provide special effects such as collecting rent, stealing properties, forcing trades, drawing extra cards, or blocking opponent actions.

4. **Building Cards:** House and Hotel cards that may only be added to fully completed color sets to boost rent income.

Notably, action cards and building cards can be placed in the bank as money, but lose their special effects once deposited.

2.3 Turn Structure and Phase Rules

Each player's turn follows three fixed phases in strict order:

Phase 1: Draw Phase

At the beginning of the turn, the player draws **2 cards** from the draw pile.

If the player's hand is empty at the start of the turn, they draw **5 cards** as a recovery mechanism.

Phase 2: Play Phase

The player may perform up to **3 card actions** per turn. Valid actions include:

1. Placing money or action cards into the bank
2. Playing property cards into the property area
3. Activating action or rent card effects
4. Attaching House or Hotel cards to completed property sets

Phase 3: End Phase & Hand Limit

After finishing actions, the player must ensure their hand size does not exceed **7 cards**.

Any extra cards must be discarded into the discard pile before the turn passes to the next player.

2.4 Economic and Interactive Rules

Payment Rules

When required to pay rent or fees, players can only use assets in their bank or property area. Cards in hand cannot be used for payment.

No-Change Rule

If the value of the card used for payment exceeds the required amount, no change is given. The extra value is forfeited.

Defense Mechanism

The *Just Say No* card can be used as an instant response to cancel any hostile action or rent attempt. This defense does not count toward the player's action limit and can be chained by multiple players.

3. Game Requirements

3.1 User Requirements

1. The system must provide a clear and user-friendly GUI that separately displays the player's hand, bank, and property zones.
2. When required to make payments, the system must allow users to choose between money cards and property cards as payment methods.
3. When targeted by hostile actions or rent effects, the system must allow players to use *Just Say No* cards to block the effect.
4. The interface must clearly show the current active player, turn status, and remaining playable actions.
5. The system must immediately detect when a player completes three property sets, end the game, and display a victory screen.

3.2 System Functional Requirements

1. The system must initialize a complete 110-card deck, assign correct attributes, and perform a random shuffle at game startup.
2. The system must automatically draw cards at the start of each turn: 2 cards normally, 5 cards if the hand is empty.
3. The system must restrict invalid operations, such as adding House or Hotel cards to incomplete property sets.
4. When a targeted action is played, the system must pause and wait for the target player to respond (defend or pay).
5. The system must correctly transfer assets between players during payment and strictly enforce the no-change rule.
6. When a player attempts to end their turn, the system must check hand size and force discarding if more than 7 cards are held.

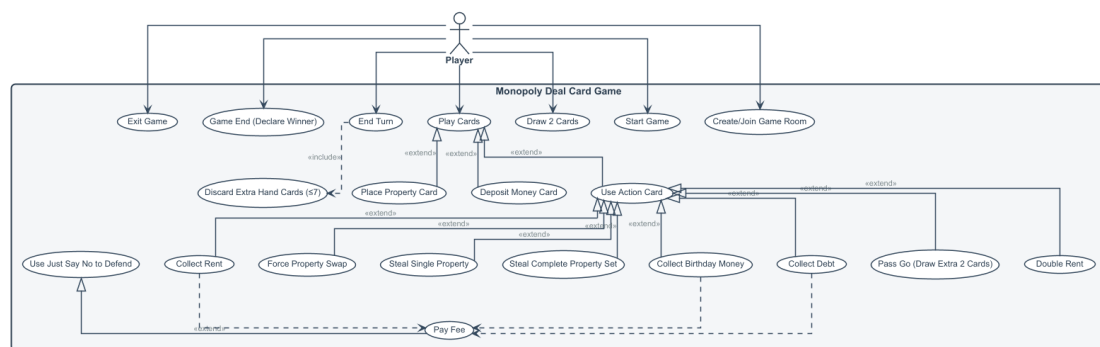
3.3 System Non-Functional Requirements

1. (Maintainability) The code must follow object-oriented principles, with high cohesion and low coupling. Method length should be controlled to ensure readability.
2. (Extensibility) The system structure should support future extensions such as new card types. The design will use interfaces and polymorphism to separate card behaviors.
3. (Performance) The GUI must respond smoothly to user operations. Interaction delay (card play, drag, turn end) must remain under 0.5 seconds.
4. (Usability) The interface layout must be intuitive, so players can quickly understand game state without excessive instructions.

4. Design idea diagram

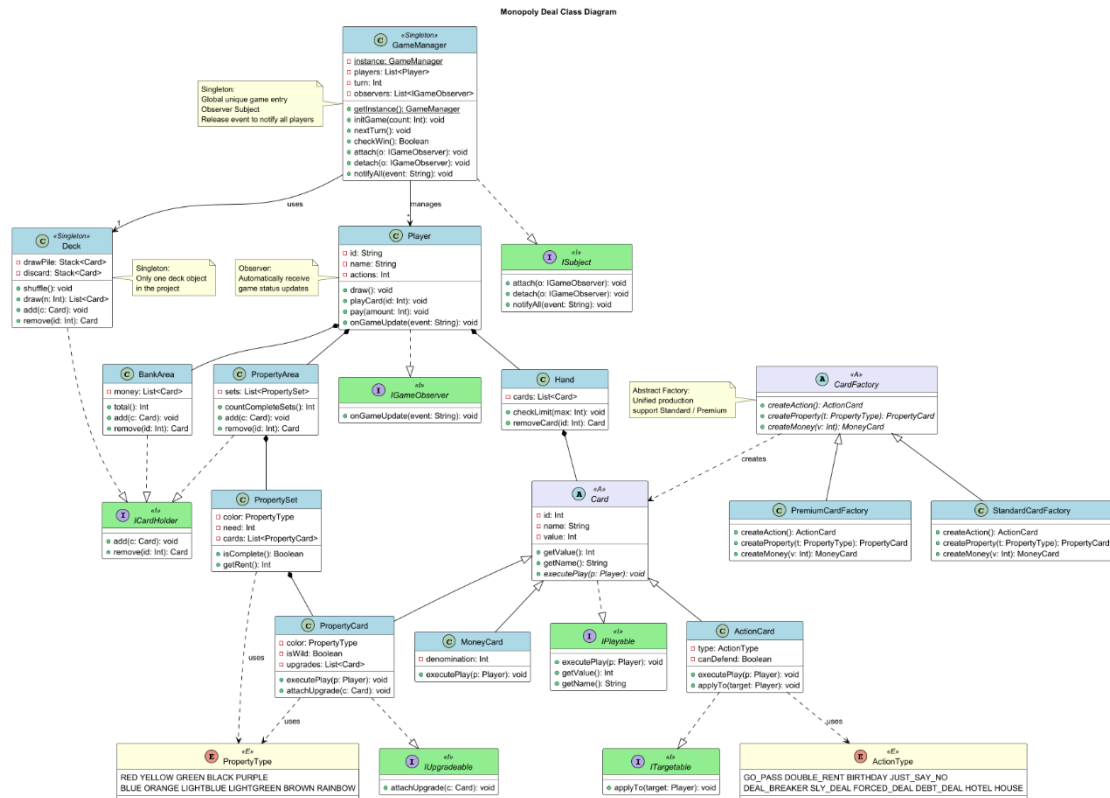
4.1 Use Case Diagram

What is shown to the project user can make the user intuitively understand the operation principle of the project and the communication relationship between classes



4.2 Class Diagram

Describe the static structure and relationship among classes, attributes, methods in the system.

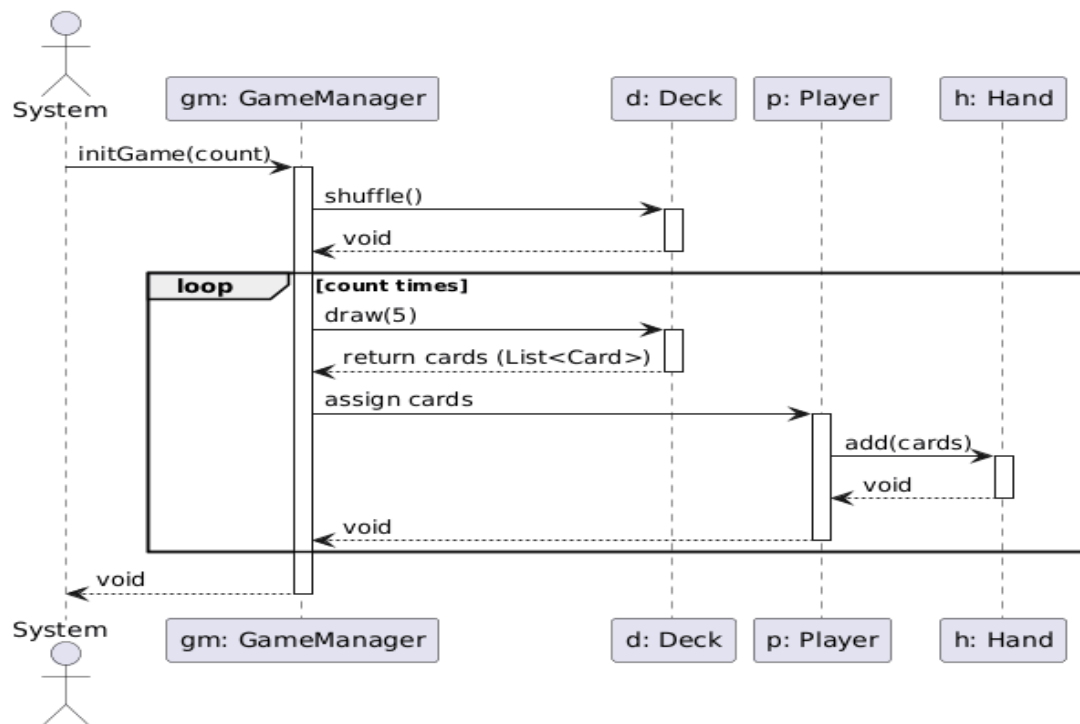


4.3 Sequence Diagram

Describes the interaction process and message passing between objects in chronological order.

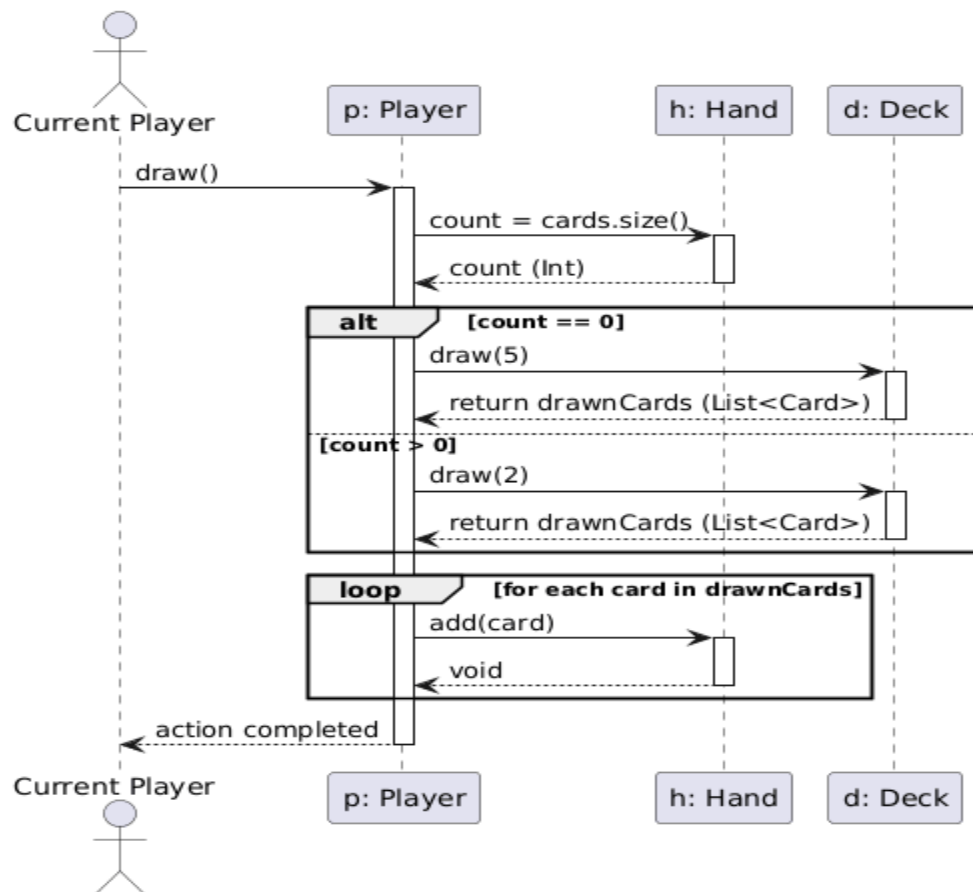
4.31 Start game

1. Start Game

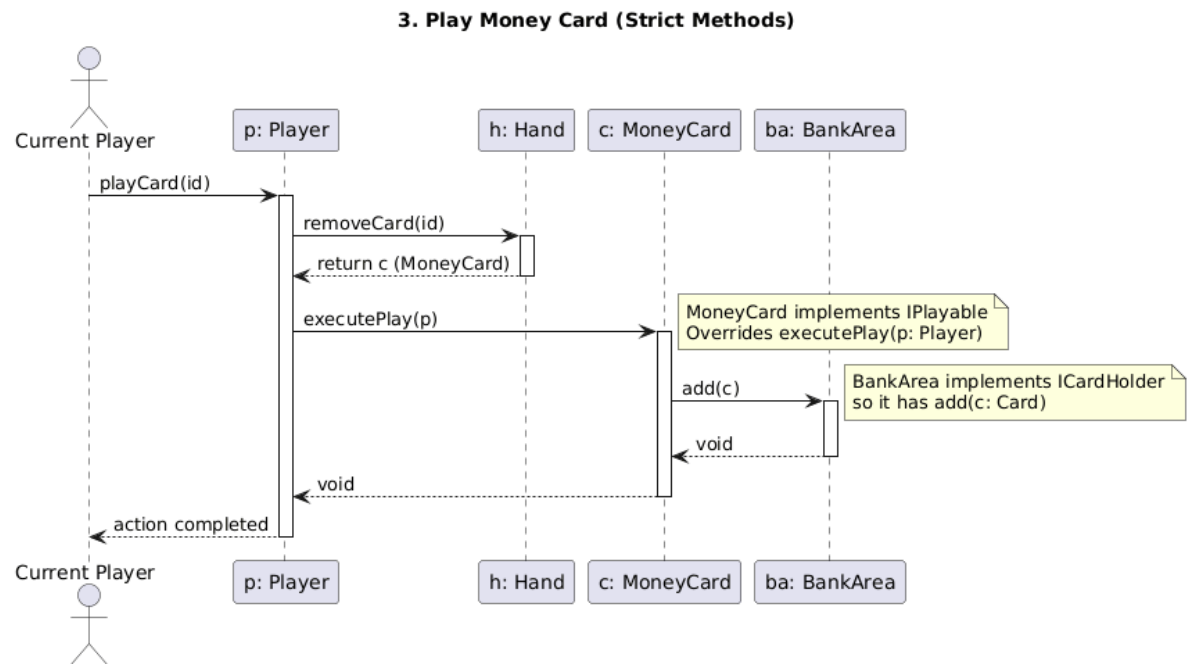


4.3.2 Turn Start - Draw Cards

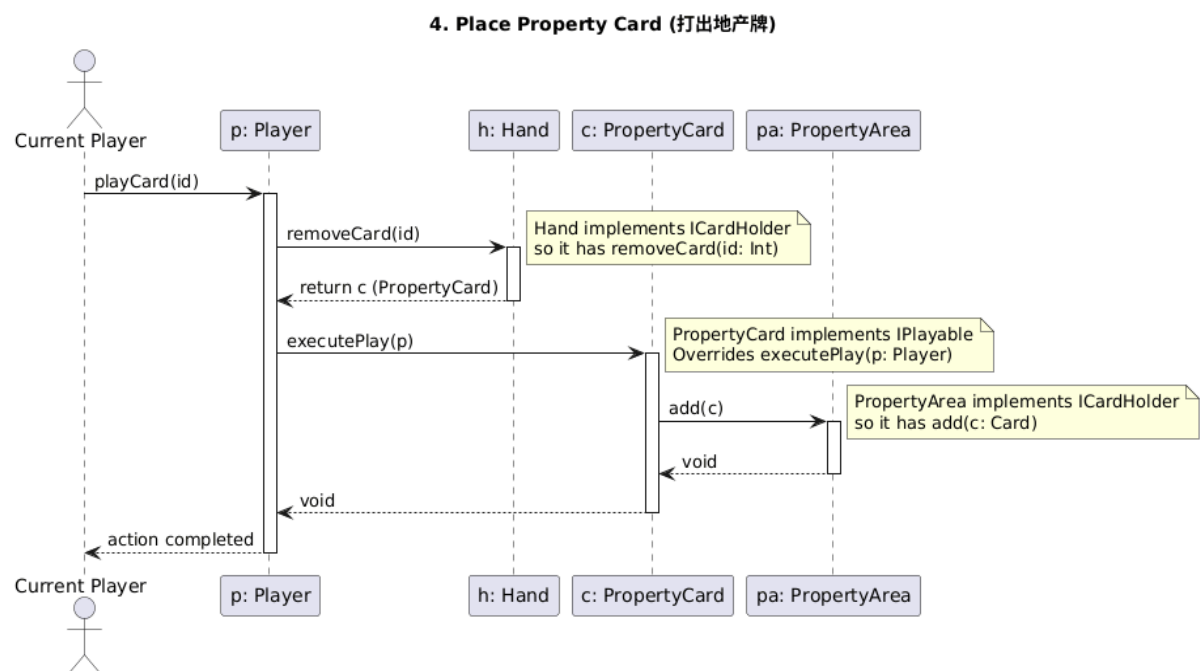
2. Turn Start - Draw Cards



4.3.3 Play Money Card (Strict Methods)

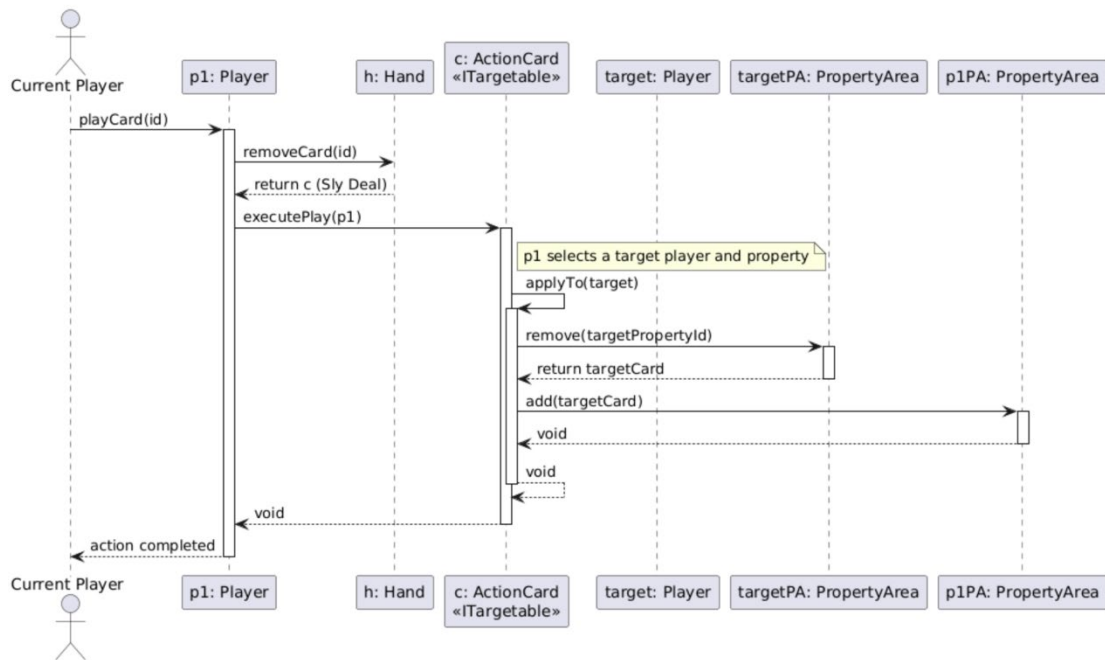


4.3.4 Place Property Card



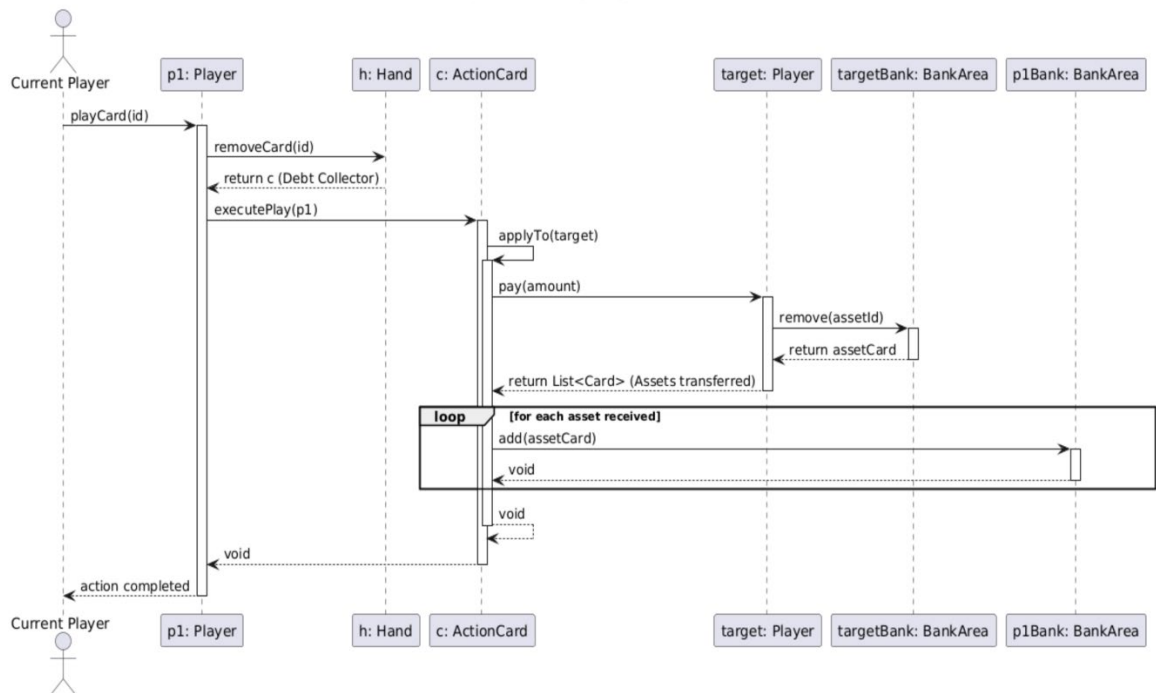
4.3.5 Play Steal Action - e.g. Sly Deal

5. Play Steal Action - e.g. Sly Deal

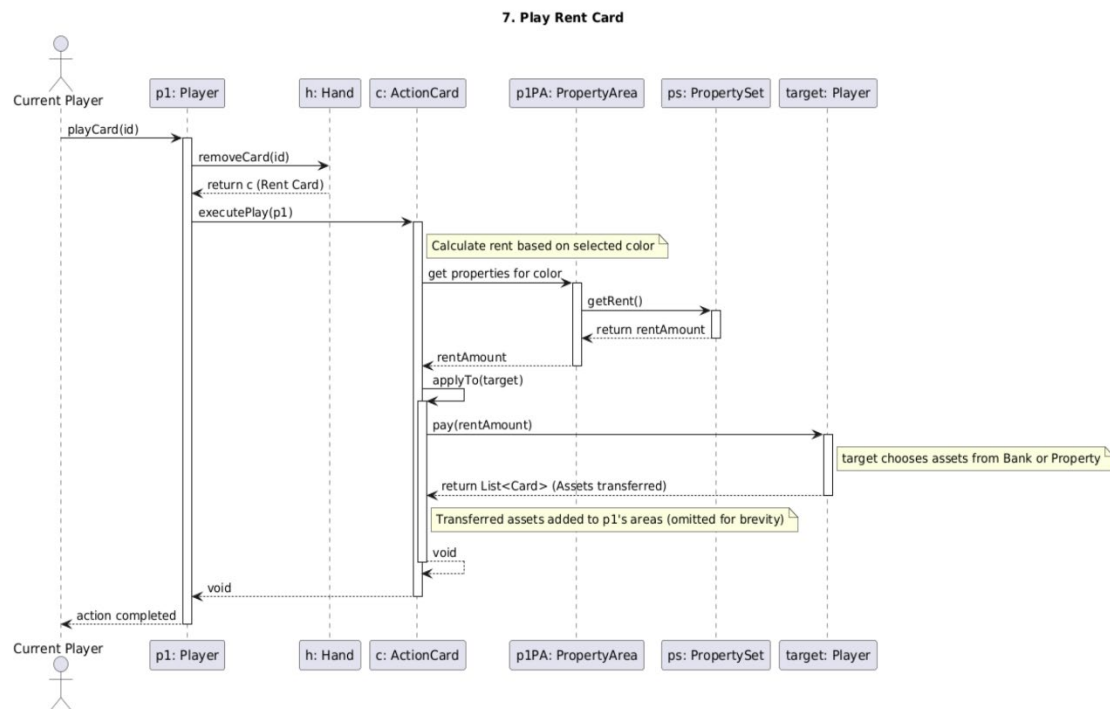


4.3.6 Play Collect Money - e.g. Debt Collector

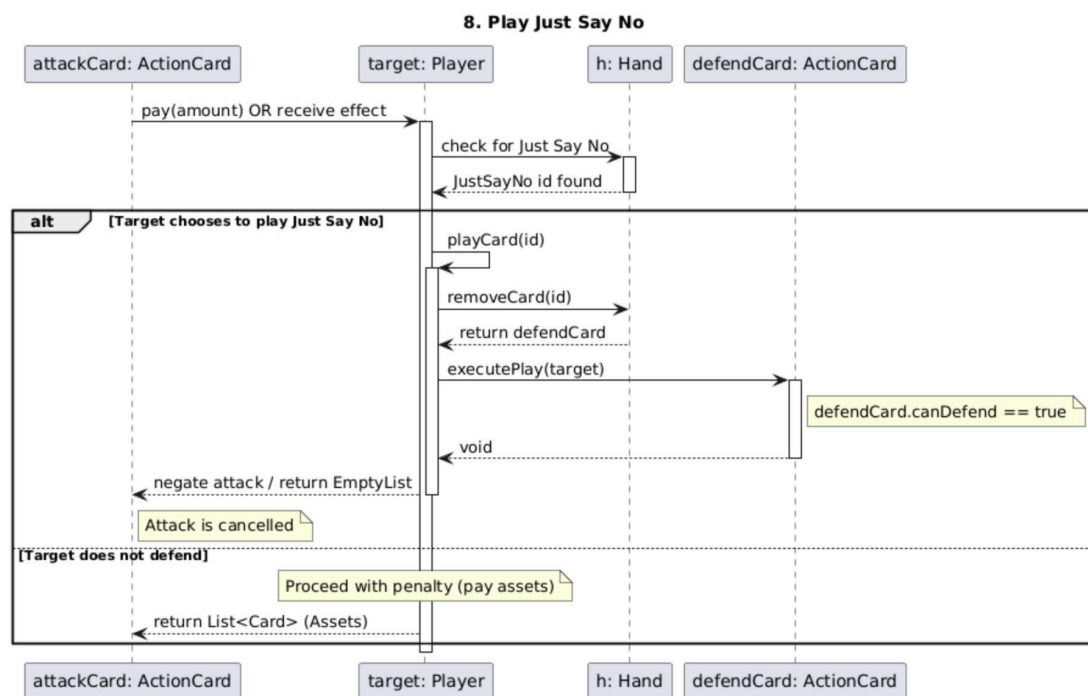
6. Play Collect Money - e.g. Debt Collector



4.3.7 Play Rent Card



4.3.8 Play Just Say No



5. Project completion timeline

Timeline	Specific Tasks
Week 7-8	Phase 1 Delivery: Complete the final PDF report.
Week 9 - 10	Data and Basic Model: 1. Create the Card class and its subclasses. 2. Initialize data for all 108 cards based on the attribute table. 3. Implement shuffle and dealing logic.
Week 10 - 11	Core Architecture and Basic Actions: 1. Implement the Player class and player actions (draw cards, deposit money). 2. Manage turn switching logic.
Week 11 - 12	Complex Algorithms and Calculations: 1. Implement rent calculation logic 2. Implement asset transfer logic (paying money or properties).
Week 12 - 13	Functional Card Implementation (Action Cards): Implement the specific effects of all 10 types of Action Cards.
Week 13	Interaction and Interface Integration: 1. Implement GUI / command-line display. 2. Add input listening (player card selection).
Week 14	Full Testing and Defensive Programming: 1. Develop test cases. 2. Capture and fix bugs.
Week 15	Final Delivery: Organize code, record demo video, and complete the

Timeline	Specific Tasks
	final project documentation.

6. Division of tasks

1.[Yang Yanzhen] 20%:

Data and Basic Model:

1. Create the Card class and its subclasses.
2. Initialize data for all 108 cards based on the attribute table.
3. Implement shuffle and dealing logic.

Functional Card Implementation (Action Cards): Implement the specific effects of all 10 types of Action Cards.

2.[Gong Chen] 20%:

Core Architecture and Basic Actions:

1. Implement the Player class and player actions (draw cards, deposit money).
2. Manage turn switching logic.

3.[Chu Tianze] 20%:

Core Architecture and Basic Actions:

1. Implement the Player class and player actions (draw cards, deposit money).
2. Manage turn switching logic.

4.[Zheng Bowei] 20%:

Interaction and Interface Integration:

1. Implement GUI / command-line display.
2. Add input listening (player card selection).

5.[Guan Jian] 20%:

Full Testing and Defensive Programming:

1. Develop test cases.
2. Capture and fix bugs.

Functional Card Implementation (Action Cards): Implement the specific effects of all 10 types of Action Cards.

6.All members:

Final Delivery: Organize code, record demo video, and complete the final project documentation.